

PDE's

- Partial differential equations are ubiquitous in nature
- Solving PDE's on computers is a huge industry
- Several different techniques are used, each with advantages/disadvantages
- Diffusion, heat flow, fluid flow, wave propagation, many many others examples of PDE's solved on computers.

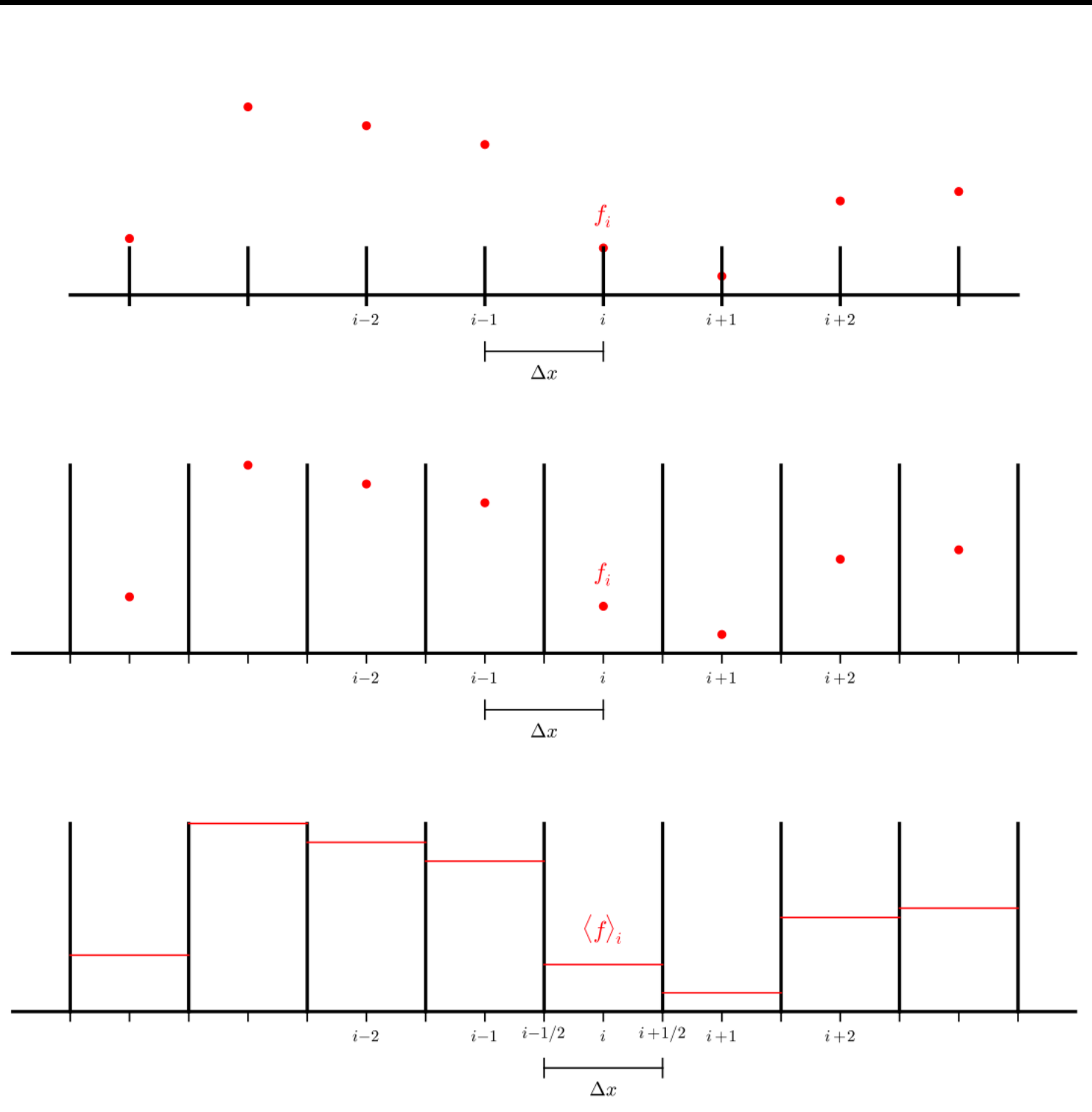
Advection Problem

- Will do fluid mechanics next. Many of the computational issues can be seen more simply through *advection*, which we will look at today.
- Imagine we have a velocity field v and a density field ρ (could be matter, could be something else).
- In advection, there are no internal forces/viscosities etc. The material just goes with the flow. Velocity is constant and field is conserved.
- Good source is tutorial from Mike Zingale, online at https://open-astrophysics-bookshelf.github.io/numerical_exercises/CompHydroTutorial.pdf

Some Techniques

- What should code even look like? Two broad classes:
- Eulerian: decompose space into domains (e.g. on a grid). Solve for $\rho(r,t)$, $v(r,t)$, etc.
 - Finite difference - function defined on grid cells
 - Finite volume - each cell covers a finite volume, value in cell is “average” of quantity across volume.
- Lagrangian: follow discrete packets of mass (“particles”) through flow
 - Smoothed particle hydrodynamics (SPH)

Eulerian Visualizations



- Top - finite difference. Function defined at grid points.
- Middle - finite difference, but with function defined at grid centers.
- Bottom - finite volume - function value is average across cell.

Figure from Zingale

Finite Volume Advection

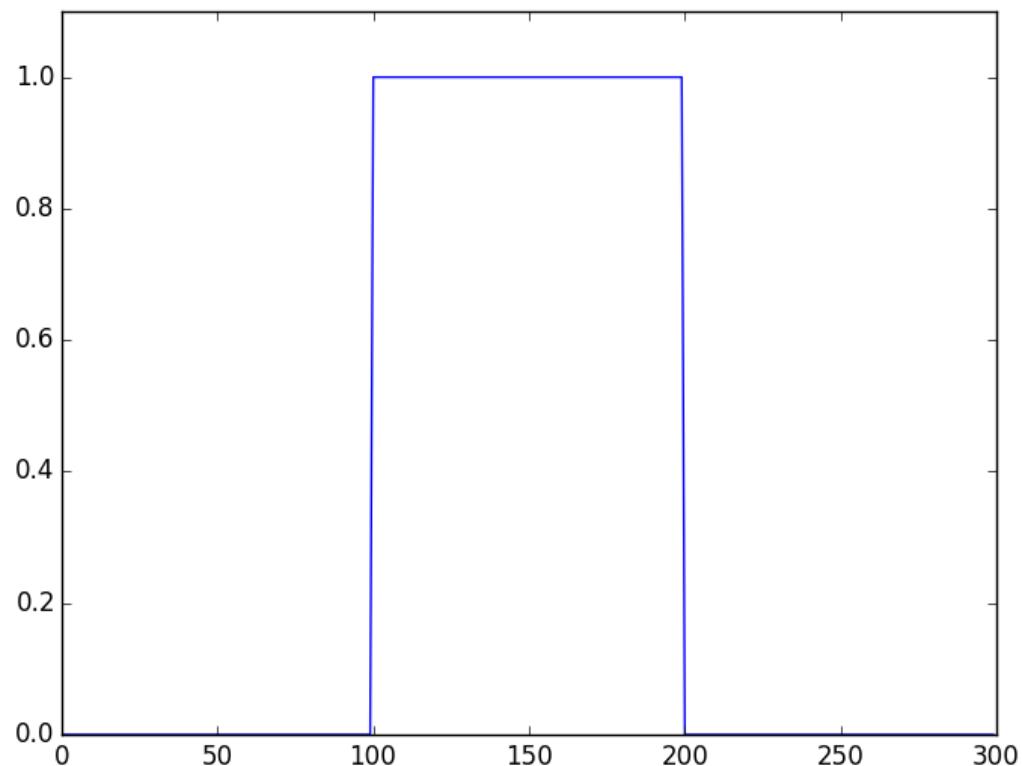
- Have density ρ_i and velocity v , with velocity taken to be uniform & constant for all grid cells.
- How does density change with time?
- Assume velocity is to the right. I flow into cell to my right, cell to my left flows into me.
- In (short time) dt flow moves vdt to the right. Cell is dx wide, so fraction of material that leaves cell is vdt/dx , total amount is $\rho_i v dt/dx$.
- Material flowing in is similarly $\rho_{i-1} vdt/dx$.
- New value is $\rho_i^{new} = \rho_i - \rho_i vdt/dx + \rho_{i-1} vdt/dx$

Finite Volume Advection

```
#simple_advect_finite_volume.py
import numpy
from matplotlib import pyplot as plt
n=300
rho=numpy.zeros(n)
rho[n/3:(2*n/3)]=1
v=1.0
dx=1.0
x=numpy.arange(n)*dx

plt.ion()
plt.clf()
plt.plot(x,rho)
```

Left: set up initial conditions. Density is 1 in the middle third of region, zero otherwise. Below left: initial density plotted. Bottom: advection code.



```
dt=1.0
for step in range(0,50):
    #take the difference in densities
    drho=rho[1:]-rho[0:-1]
    #update density. We haven't said what happens at
    #cell 0 (since cell -1 doesn't exist), ignore for now
    rho[1:]=rho[1:]-v*dt/dx*drho
    plt.clf()
    plt.plot(x,rho)
    plt.draw()
```


Conservation

- New value is $\rho_i^{\text{new}} = \rho_i - \rho_i v dt/dx + \rho_{i-1} v dt/dx$
- But cell $i+1$ looks the same, with $i \rightarrow i+1$: $\rho_{i+1}^{\text{new}} = \rho_{i+1} - \rho_{i+1} v dt/dx + \rho_i v dt/dx$
- if I sum - $\rho_i^{\text{new}} + \rho_{i+1}^{\text{new}} = \rho_i - \rho_i v dt/dx + \rho_{i-1} v dt/dx + \rho_{i+1} - \rho_{i+1} v dt/dx + \rho_i v dt/dx$
- Amount leaving me matches amount flowing into neighbour: $\rho_i^{\text{new}} + \rho_{i+1}^{\text{new}} = \rho_i + \rho_{i+1} - (\rho_{i+1} - \rho_{i-1}) v dt/dx$
- If I sum over all cells, cancellation continues: $\sum \rho^{\text{new}} = \sum \rho - (\rho_{\text{end}} - \rho_{\text{begin}}) v dt/dx$
- Modulo funny things at edges, stuff is conserved. This is a good thing.

Differential Form

- Say we have a conserved flow, now with non-constant velocity.
- Amount flowing out in dt is $v_r \rho_r$. Amount flowing in is $v_l \rho_l$. Net amount is $-\partial(v\rho)/\partial x$. If flow is conserved, $\partial\rho/\partial t = -\partial(v\rho)/\partial x$ or $\partial\rho/\partial t + \partial(v\rho)/\partial x = 0$. This form is very general, we will see it more in fluids.
- In general, we can have multiple dimensions. In this case, the x -derivative becomes a divergence: $\partial\rho/\partial t + \nabla \cdot (\rho\mathbf{v}) = 0$
- For advection, velocity is constant so can pull out. Equation we're really solving is: $\partial\rho/\partial t + v\partial\rho/\partial x = 0$

Boundary Conditions

- For a finite-sized region, we have no way of solving for what happens at domain boundary.
- We need to specify this behaviour as part of the problem.
- One common case is all gradients equal zero on boundary
- Another common case is *periodic*: $\rho_{-1} = \rho_{\text{end}}$.
- What would our advection example look like with periodic boundary conditions?
- You should *always* think carefully about your boundary conditions.

Guard Cells

- The way BC's are implemented in practice is through *guard* or *ghost* cells.
- Pad your domain with extra cells. Fill them in as per BC's. Take time step. Extract original domain.
- # of guard cells may depend on details of your algorithm, but you will almost certainly need them.

In Practice

```
#advect_finite_volume_guard.py
dt=1.0
for step in range(0,150):
    #we need one guard cell - make a 1-larger temp array
    big_rho=numpy.zeros(n+1)
    big_rho[1:]=rho
    #explicitly set the density of the guard cell
    big_rho[0]=0
    #take the difference in densities
    drho=big_rho[1:]-big_rho[0:-1]
    big_rho[1:]=big_rho[1:]-v*dt/dx*drho
    rho=big_rho[1:]
    plt.clf()
    plt.axis([0,n,0,1.1])
    plt.plot(x,rho)
    plt.draw()
```

```
#advect_finite_volume_guard_compact.py
dt=1.0
#set up padded array outside loop
big_rho=numpy.zeros(n+1)
big_rho[1:]=rho
del rho #we can delete the to save space
for step in range(0,150):
    #still need to explicitly set the density of the guard cell
    big_rho[0]=0
    #take the difference in densities
    drho=big_rho[1:]-big_rho[0:-1]
    big_rho[1:]=big_rho[1:]-v*dt/dx*drho

    plt.clf()
    plt.axis([0,n,0,1.1])
    plt.plot(x,big_rho[1:])
    plt.draw()
```

- Initialization is identical.
- For simple advection need one extra cell.
- Can even do in-place, saving memory, probably faster, too (see bottom)

Time Steps

- Smaller time step normally more accurate.
- Let's look at solution for some different time steps.
- What happened?
- Behaviour of sharp features often very important - in practice, run test problems with known solutions to verify behaviour.

```
#advect_finite_volume_timestep.py
dt=1.0
big_rho=numpy.zeros(n+1)
big_rho[1:]=rho
del rho #we can delete the to save space
oversamp=10 #let's do finer timestamps
dt_use=dt/oversamp
for step in range(0,150):

    big_rho[0]=0
    for substep in range(0,oversamp):
        drho=big_rho[1:]-big_rho[0:-1]
        big_rho[1:]=big_rho[1:]-v*dt_use/dx*drho

plt.clf()
plt.axis([0,n,0,1.1])
plt.plot(x,big_rho[1:])
plt.draw()
```

Now What Happens With Big Timestep?

- Try this and see what happens.
- Whoa...

```
#advect_finite_volume_timestep_coarse.py
dt=1.0
big_rho=numpy.zeros(n+1)
big_rho[1:]=rho
del rho #we can delete the to save space
oversamp=0.5 #let's do coarser timestamps
dt_use=dt/oversamp
for step in range(0,150):

    big_rho[0]=0
    drho=big_rho[1:]-big_rho[0:-1]
    big_rho[1:]=big_rho[1:]-v*dt_use/dx*drho

    plt.clf()
    plt.axis([0,n,0,1.1])
    plt.plot(x,big_rho[1:])
    plt.draw()
```

Stability

$$\rho_j^{\text{new}} = \rho_j - (\rho_j - \rho_{j-1})vdt/dx$$

- You can learn a lot by plugging in sine waves.
- If $\rho_j = \exp(ikj)$, $\rho_j^{\text{new}} = \text{what?}$ define $a = vdt/dx$
- $\rho_j^{\text{new}} = \exp(ikj) - a(\exp(ikj) - \exp(ik(j-1))) = \exp(ikj) - a(\exp(ikj) - \exp(-ik)\exp(ikj))$
- $\rho_j^{\text{new}} = \exp(ikj) * [1 - a(1 - \exp(-ik))]$
- If magnitude of quantity in [] gets bigger than unity, solution will grow with time. Our code would be *unstable* - this is bad!

CFL Condition ($a=vdt/dx$)

- Look at $1-a(1-\exp(-ik))$. $1-\exp(-ik)$ is bounded by $(0,2)$
- if $0 \leq a \leq 1$, solution always stable.
- if $a > 2$, then $\lambda = 1-2a$ can have magnitude > 1 for sufficiently large a .
- By construction, a is positive, so can't get $\lambda > 1$. But can get $\lambda < -1$: $1-2a < -1$, $2 < 2a$, or $a > 1$.
- For stability, $a \leq 1$, or $dt \leq dx/v$. In words, dt has to be shorter than crossing time for cell.
- This is called the Courant–Friedrichs–Lewy (CFL) condition. vdt/dx is the Courant number.

2nd Order

- Could we do better with higher-order derivative?
- Try $\partial \rho / \partial x = (\rho_{j+1} - \rho_{j-1}) / 2\Delta x$.
- $\rho_{x,t+1} = \rho_{x,t} + \alpha(\rho_{x+1,t} - \rho_{x-1,t}) / 2$.
- let $\rho_{x,t} = \exp(ikx)$. Plug in: $\exp(ikx) + \alpha(\exp(ik(x+1)) - \exp(ik(x-1))) / 2$
 $= \exp(ikx)(1 + i\alpha \sin(k))$
- Magnitude larger than 1 for $k \neq 0$. Always unstable, no matter how small we make α . CFL condition *necessary*, but not sufficient.

Numerical Viscosity/Lax

- We saw setting df/dx with $(f_{x+1}-f_{x-1})/2dx$ led to unconditional instability in advection.
- However - take $(f(x,t+dt)-f(x,t))/dt$ for time derivative to $(f(x,t+dt)-(f(x+dx,t)+f(x-dx,t))/2)/dt$ leads to stability. Can you guess criterion for stability?
- Rewrite: $(f(x,t+dt)-f(x,t))/dt = -v(f(x+dx,t)-f(x-dx,t))/2dx + (f(x+dx,t)-2f(x,t)+f(x-dx,t))/2dt$.
- This is solving $df/dt = -vdf/dx + (dx)^2/2dt \nabla^2 f$. New term looks like diffusion/viscosity equations - we're adding numerical viscosity to induce stability.

Lagrangian

- An alternative way of solving is to label fluid packets, then follow them with time.
- Labelling usually refers to position at time $t=0$.
- Particularly simple for advection: $x^{\text{new}}=x+vdt$, or $x_j(t)=j+vt$

In Practice

```
#advect_lagrangian.py
import numpy
from matplotlib import pyplot as plt

n=300
#set up density the usual way
rho=numpy.zeros(n)
rho[n/3:(2*n/3)]=1

v=1.0
dx=1.0
x=numpy.arange(n)*dx

plt.ion()
plt.clf()
plt.axis([0,n,0,1.1])
plt.plot(x,rho)
plt.draw()

dt=1.0
#now take time steps
for step in range(0,150):
    #new particle position is just old position plus velocity
    x=x+v*dt
    plt.clf()
    plt.axis([0,1.5*n,0,1.1])
    plt.plot(x,rho, '*')
    plt.draw()
```

- Note differences in code - we just find new x position.
- Since we only follow particles that existed at beginning, we can ignore boundary conditions.

Eulerian vs. Lagrangian

- Eulerian vs. Lagrangian choice can depend on problem
- Mass conservation trivial with Lagrangian codes.
- More work to calculate density in Lagrangian code
- Lagrangian codes can have multiple velocities at same position. Unnatural with Eulerian code.
- In astrophysics, streams of dark matter can cross - Lagrangian might work better. Streams of gas can't (the wind only blows in one direction) so Eulerian might be simpler there.